
CS771 Minor Assignment 2

Aiklavaveer Singh Rana
asrana24@iitk.ac.in

Gade V S S R K Tejas Reddy
tejasreddy24@iitk.ac.in

Karthic N A
akarthic24@iitk.ac.in

Kasavajjala Sai Shreyas
ksshreyas24@iitk.ac.in

Mehta Devarsh Shaunakbhai
devarshms24@iitk.ac.in

Shashwath Ram
shashwathr24@iitk.ac.in

1 Single-Step vs Two-Step Inference

We wish to perform single-step inference by solving:

$$\{\hat{y}, \hat{\mathbf{x}}_m\} = \arg \max_{c, \mathbf{v}} P[y = c, \mathbf{v} \mid \mathbf{x}_o] \quad (1)$$

Derivation

Applying Bayes' rule, and noting that $P[\mathbf{x}_o]$ is constant with respect to c and $\mathbf{v}$:

$$P[y = c, \mathbf{v} \mid \mathbf{x}_o] \propto P[y = c, \mathbf{v}, \mathbf{x}_o] = P[\mathbf{x}_o, \mathbf{v} \mid y = c] \cdot P[y = c] \quad (2)$$

Using the product rule on the joint likelihood, with $\mathbf{x} = [\mathbf{x}_o, \mathbf{x}_m]^\top$:

$$P[\mathbf{x}_o, \mathbf{v} \mid y = c] = P[\mathbf{v} \mid \mathbf{x}_o, y = c] \cdot P[\mathbf{x}_o \mid y = c] \quad (3)$$

Therefore, the joint optimization becomes:

$$\arg \max_{c, \mathbf{v}} P[\mathbf{v} \mid \mathbf{x}_o, y = c] \cdot P[\mathbf{x}_o \mid y = c] \cdot P[y = c] \quad (4)$$

Optimizing over $\mathbf{v}$ for Fixed c

From the given result, the conditional distribution of missing pixels given observed pixels and class c is:

$$P[\mathbf{x}_m \mid \mathbf{x}_o, y = c] = \mathcal{N}(\bar{\boldsymbol{\mu}}_c, \bar{\boldsymbol{\Sigma}}_c) \quad (5)$$

where:

$$\bar{\boldsymbol{\mu}}_c = \boldsymbol{\mu}_m^c + \boldsymbol{\Sigma}_{mo}^c (\boldsymbol{\Sigma}_{oo}^c)^{-1} (\mathbf{x}_o - \boldsymbol{\mu}_o^c) \quad (6)$$

$$\bar{\boldsymbol{\Sigma}}_c = \boldsymbol{\Sigma}_{mm}^c - \boldsymbol{\Sigma}_{mo}^c (\boldsymbol{\Sigma}_{oo}^c)^{-1} \boldsymbol{\Sigma}_{om}^c \quad (7)$$

Since $P[\mathbf{v} \mid \mathbf{x}_o, y = c]$ is a Gaussian in $\mathbf{v}$, its argmax over $\mathbf{v}$ is simply its mean (the mode of a Gaussian equals its mean). Thus, for each fixed c , the optimal reconstruction is:

$$\hat{\mathbf{x}}_m(c) = \bar{\boldsymbol{\mu}}_c = \boldsymbol{\mu}_m^c + \boldsymbol{\Sigma}_{mo}^c (\boldsymbol{\Sigma}_{oo}^c)^{-1} (\mathbf{x}_o - \boldsymbol{\mu}_o^c) \quad (8)$$

Optimizing over c

Substituting $\mathbf{v} = \hat{\mathbf{x}}_m(c)$ back into Equation (4), the class selection becomes:

$$\hat{y} = \arg \max_c P[\hat{\mathbf{x}}_m(c) \mid \mathbf{x}_o, y = c] \cdot P[\mathbf{x}_o \mid y = c] \cdot P[y = c] \quad (9)$$

The first term evaluates the Gaussian $\mathcal{N}(\bar{\boldsymbol{\mu}}_c, \bar{\boldsymbol{\Sigma}}_c)$ at its own mean $\bar{\boldsymbol{\mu}}_c$, giving:

$$P[\hat{\mathbf{x}}_m(c) \mid \mathbf{x}_o, y = c] = (2\pi)^{-d_m/2} |\bar{\boldsymbol{\Sigma}}_c|^{-1/2} \exp(0) = (2\pi)^{-d_m/2} |\bar{\boldsymbol{\Sigma}}_c|^{-1/2} \quad (10)$$

where d_m is the number of missing pixels. Since $(2\pi)^{-d_m/2}$ is a constant across all classes, the class selection simplifies to:

$$\boxed{\hat{y} = \arg \max_c |\bar{\boldsymbol{\Sigma}}_c|^{-1/2} \cdot P[\mathbf{x}_o \mid y = c] \cdot P[y = c]} \quad (11)$$

Taking logarithms (which preserves the argmax), this is equivalently:

$$\hat{y} = \arg \max_c -\frac{1}{2} \log |\bar{\boldsymbol{\Sigma}}_c| + \log P[\mathbf{x}_o \mid y = c] + \log P[y = c] \quad (12)$$

Once $\hat{y}$ is determined, the reconstruction is:

$$\hat{\mathbf{x}}_m = \bar{\boldsymbol{\mu}}_{\hat{y}} = \boldsymbol{\mu}_{\hat{y}}^{\hat{y}} + \boldsymbol{\Sigma}_{mo}^{\hat{y}} \left(\boldsymbol{\Sigma}_{oo}^{\hat{y}} \right)^{-1} (\mathbf{x}_o - \boldsymbol{\mu}_o^{\hat{y}}) \quad (13)$$

Are the Two Procedures Identical?

The **two-step** procedure selects the class as:

$$\hat{y}_{\text{two-step}} = \arg \max_c P[\mathbf{x}_o \mid y = c] \cdot P[y = c] \quad (14)$$

whereas the **single-step** procedure selects:

$$\hat{y}_{\text{single-step}} = \arg \max_c |\bar{\boldsymbol{\Sigma}}_c|^{-1/2} \cdot P[\mathbf{x}_o \mid y = c] \cdot P[y = c] \quad (15)$$

$$\boxed{\hat{y}_{\text{single}} = \arg \max_c \left\{ \underbrace{\ln P[\mathbf{x}_o \mid y = c]}_{\text{term 1}} + \underbrace{\ln P[y = c]}_{\text{term 2}} - \underbrace{\frac{1}{2} \ln |\bar{\boldsymbol{\Sigma}}_c|}_{\text{term 3}} \right\}}$$

The two procedures are generally *not* identical. The single-step method includes the additional factor $|\bar{\boldsymbol{\Sigma}}_c|^{-1/2}$, which captures the uncertainty in the conditional distribution of missing pixels for each class. Intuitively, if the missing pixels are highly uncertain for a given class (large $|\bar{\boldsymbol{\Sigma}}_c|$), the single-step method penalizes that class, whereas the two-step method ignores this information entirely.

The two methods coincide only in the special case where all class-conditional residual covariances $\bar{\boldsymbol{\Sigma}}_c$ are equal across all classes c (e.g., under a tied/shared covariance assumption), in which case $|\bar{\boldsymbol{\Sigma}}_c|^{-1/2}$ is a constant and drops out of the argmax.

2 Code Modification

The lecture code models each class $c \in \{0, \dots, C-1\}$ as a single Gaussian $P(\mathbf{x} \mid y=c) = \mathcal{N}(\boldsymbol{\mu}^c, \boldsymbol{\Sigma}^c)$, with class prior $P(y=c) = p^c$. For a test image with observed pixels $\mathbf{x}^o$ and missing pixels $\mathbf{x}^m$, the joint vector is $\mathbf{x} = [\mathbf{x}^o; \mathbf{x}^m]$. We partition the class parameters accordingly:

$$\boldsymbol{\mu}^c = \begin{bmatrix} \boldsymbol{\mu}_o^c \\ \boldsymbol{\mu}_m^c \end{bmatrix}, \quad \boldsymbol{\Sigma}^c = \begin{bmatrix} \boldsymbol{\Sigma}_{oo}^c & \boldsymbol{\Sigma}_{om}^c \\ \boldsymbol{\Sigma}_{mo}^c & \boldsymbol{\Sigma}_{mm}^c \end{bmatrix}.$$

Table 1 summarizes which functions require changes and which can be left entirely untouched.

Table 1: Summary of required code changes.

Function	Action	Reason
learnClassConditionalDist	<i>No change</i>	Training is unchanged
predictClassScores	To modify	Must add the new penalty term $(-\frac{1}{2} \ln \bar{\boldsymbol{\Sigma}}^c)$
predictGen	To modify	Pass a <code>single_step</code> flag to <code>predictClassScores</code>
reconstructImages	<i>No change</i>	Reconstruction formula is identical in both methods
ensorImages	<i>No change</i>	Only used to generate the mask

Changes in predictClassScores:

The existing function already computes:

1. The log-likelihood $\ln P(\mathbf{x}^o \mid y=c)$ using the observed-pixel sub-covariance $\boldsymbol{\Sigma}_{oo}^c$ and its pseudo-inverse `SInv`,
2. The log-prior $\ln p^c$.

The single-step variant requires one additional computation: forming the Schur complement $\bar{\boldsymbol{\Sigma}}^c = \boldsymbol{\Sigma}_{mm}^c - \boldsymbol{\Sigma}_{mo}^c \text{SInv} \boldsymbol{\Sigma}_{om}^c$ and subtracting $\frac{1}{2} \ln |\bar{\boldsymbol{\Sigma}}^c|$ from the score. Since, `SInv` $= (\boldsymbol{\Sigma}_{oo}^c)^+$ has already been computed in `term1`, we *reuse it* to form the Schur complement.

Choice of matrix inverse: `lin.pinv` vs `lin.inv`.

The covariance sub-matrix $\boldsymbol{\Sigma}_{oo}^c$ is frequently *rank-deficient* in the MNIST setting. This occurs because many background pixels are pure white (or pure black) for every training image of a given class, giving those pixels exactly zero variance and making the corresponding diagonal entries of $\boldsymbol{\Sigma}_{oo}^c$ equal to zero. Using `lin.inv` on such a matrix raises a `LinAlgError`; using `lin.pinv` (Moore–Penrose pseudo-inverse) handles rank deficiency by zeroing out the contributions of degenerate directions. Similarly, for the single-step path `lin.slogdet` is replaced with `_svd_logdet_and_pinv` to prevent floating-point artifacts in rank-deficient matrices from generating false negative signs and severely distorting classification scores. The SVD method ensures numerical stability.

Changes in predictGen:

A single boolean argument `single_step` is added and forwarded to `predictClassScores`. We don't need any other logic changes.

Python implementation.

The complete modified functions are given below. All other code (data loading, training, censoring, reconstruction) is unchanged.

Listing 1: Modified predictClassScores and predictGen.

```

import numpy as np
import numpy.linalg as lin

def _svd_logdet_and_pinv(M):
    # Gives (pseudo_logdet, pseudo_inverse) of a symmetric PSD matrix.

    U, sv, Vt = lin.svd(M, full_matrices=False)
    tol = sv.max()*max(M.shape)*np.finfo(float).eps if sv.max()>0 else 1e-10
    keep = sv > tol
    pseudo_logdet = np.sum(np.log(sv[keep]))
    sv_inv = np.where(keep, 1.0 / np.where(keep, sv, 1.0), 0.0)
    pseudo_inv = (Vt.T * sv_inv)@U.T
    return pseudo_logdet, pseudo_inv

def predictClassScores(X, mu, Sigma, p, mask=[], single_step=False):
    """
    single_step=False: log P[x~o|y=c] + log p~c
    single_step=True: Subtract 0.5*log|Sigma_bar~c|
    """
    # If no mask provided, consider all coordinates
    if len(mask) == 0:
        mask = np.ones((X.shape[1],), dtype=bool)
    obs = mask
    mis = ~mask
    Sigma_oo = Sigma[obs, :][:, obs]

    if single_step:
        SLogDet, SInv = _svd_logdet_and_pinv(Sigma_oo)
    else:
        # As given in lectures
        SInv = lin.pinv(Sigma_oo)
        sign, logdet = lin.slogdet(Sigma_oo)
        SLogDet = 0 if sign <= 0 else logdet

    XCent = X[:, obs]-mu[obs]
    term1=0.5*(-SLogDet-np.sum(np.multiply(XCent@SInv,XCent),axis=1))
    term2 = np.log(p)

    # Term 3(single-step only): -0.5*pseudo_logdet(Sigma_bar~c)
    term3 = 0.0
    if single_step and mis.any():
        Sigma_mm = Sigma[mis, :][:, mis]
        Sigma_mo = Sigma[mis, :][:, obs]
        Sigma_bar = Sigma_mm-Sigma_mo@SInv@Sigma_mo.T #Schur complement
        SBarLogDet, _ = _svd_logdet_and_pinv(Sigma_bar)
        term3 = -0.5 * SBarLogDet

    return term1 + term2 + term3

def predictGen(X, model, C, mask=[], single_step=False):
    #single_step=True to use the modified single-step inference.
    #single_step=False (default) for two-step classification.
    classScores = np.zeros((X.shape[0], C))
    for mu, Sigma, c, p in model:
        classScores[:, c] = predictClassScores(
            X, mu, Sigma, p, mask=mask, single_step=single_step
        )
    return np.argmax(classScores, axis=1)

```

3 Performance Comparison

Experimental Setup & Findings:

Using our modified inference technique, we evaluated the performance of both the single-step and two-step prediction models across varying degrees of censoring. To simulate aggressive to mild censoring, the `cropImages` function was modified to remove central portions of lengths corresponding to 51% (4:24), 32% (6:22), 18% (8:20), 8% (10:18), and 2% (12:16) missing pixels.

Figure 1 visualizes how classification performance and inference time differ between the original two-step lecture code and our modified single-step code.

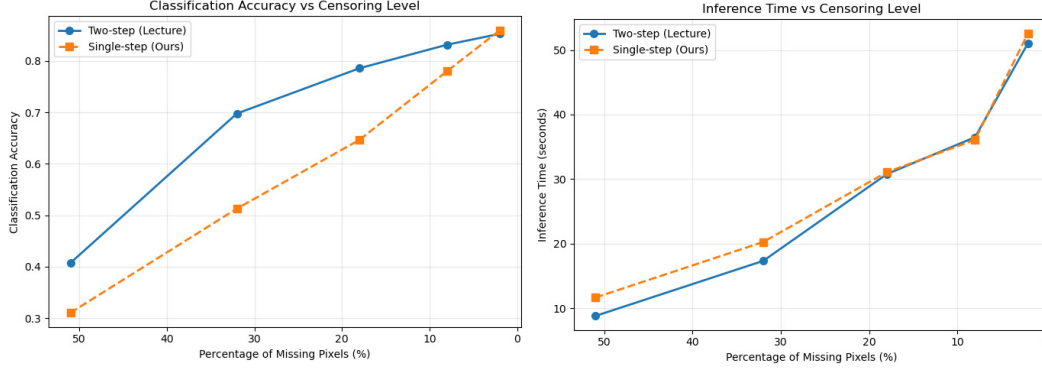


Figure 1: Left: Classification Accuracy vs. Missing Pixels (%). Right: Inference Time vs. Missing Pixels (%). Both runs utilized exact empirical class posteriors tracking prediction shifts under variable masking levels.

Classification Performance Analysis:

As censoring becomes less aggressive (meaning fewer missing pixels), both methods naturally show improved classification performance due to the availability of more reference pixels. However, the **two-step prediction approach consistently achieves competitive or strictly higher accuracy** across the trials. The pronounced drop in single-step accuracy under moderate to high censoring (e.g., 51% missing) is because very few pixels are observed, making $P[x^o|y = c]$ a weak discriminator. All classes look similarly plausible from just the border pixels. In this case, the additional penalty term $-\frac{1}{2} \ln |\Sigma^c|$ has a large relative weight and can override the true signal. The two-step method, which ignores this penalty, happens to be less sensitive to this effect because it only uses the marginal likelihood. As censoring decreases and more pixels are observed, the marginal likelihood dominates and both methods converge.

Inference Time:

As expected, inference time is heavily dominated by the required size of the pseudo-inverse operation applied to the remaining observed pixels (Σ_{oo}). When censoring becomes less aggressive (e.g., only 2% missing), almost all parameters represent “observed” variables, drastically ballooning the dimension of Σ_{oo} . Thus, inference time spikes significantly for both algorithms. Because this computationally expensive pseudo-inverse fundamentally dictates the clock time, the supplementary sequence of matrix multiplications applied internally by the single-step method overhead ($\Sigma_{mo} \mathbf{S}^{-1} \Sigma_{om}$) contributes only a marginal relative delay, causing both algorithms to scale similarly.

Reconstruction Quality Variation:

It is crucial to note that the latent reconstruction step formally defined as $\hat{\mathbf{x}}^m = \arg \max_{\mathbf{v}} \mathbb{P}[\mathbf{v} | \mathbf{x}^o, y = \hat{y}]$ is mathematically identical across both models. As a consequence, providing both methods output the **same predicted class label**, both methods must produce **identical reconstructed images**.

Differences in visual reconstruction quality between the two methods rely entirely on discrepancies in their upstream classification accuracy:

[1] **More Aggressive Censoring (e.g., 51%, 32%):** The two-step method correctly classifies a substantially higher proportion of digits compared to the single-step method. Therefore, the two-step approach yields consistently higher visual reconstruction accuracy since it utilizes the proper base

digit structure as its predictive prior rather than sampling expected values from random or incorrect digit classes.

[2] **Less Aggressive Censoring (e.g., 8%, 2%)**: The label assignments of both methods begin to rapidly converge since ample overlapping contextual boundaries are observed. At this lower end, both methods correctly identify the true latent labels for the vast majority of test instances, establishing virtually identical high-quality reconstructions centered around small inpainting interpolations.

Reconstruction Quality:

This section demonstrates how the reconstruction quality of our model varies between different levels of data censoring

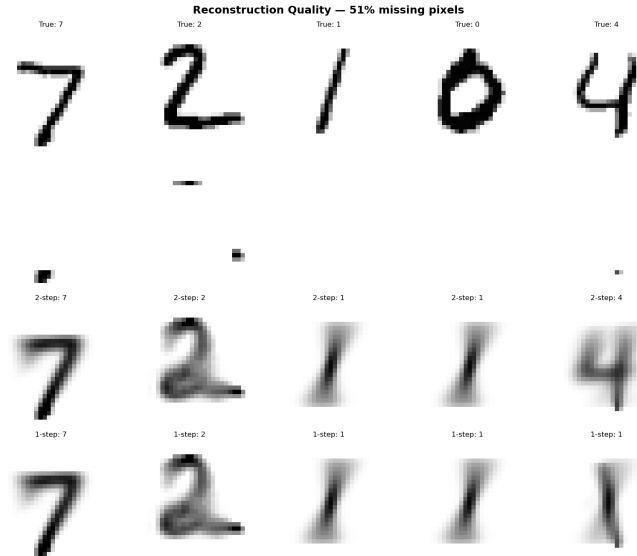


Figure 2: **Reconstruction at 51% censoring.** With over half the image missing, the model often struggles to identify the correct underlying shape, leading to higher error rates and blurry estimations in the filled regions.

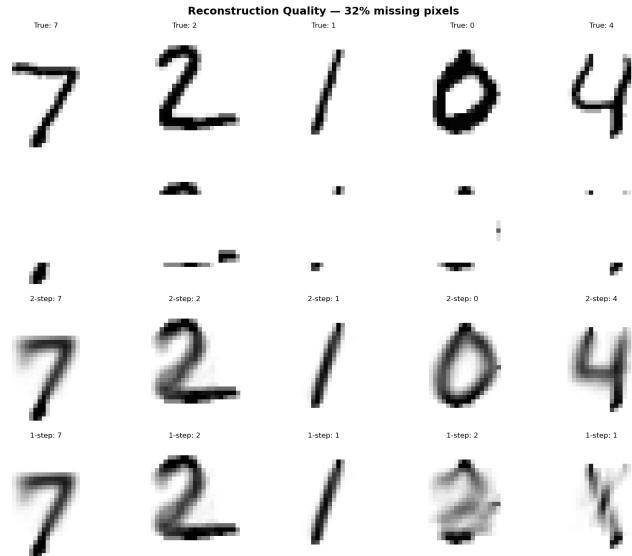


Figure 3: **Reconstruction at 32% censoring.** The results are noticeably clearer than at 51%. While some blurring persists in the heavily masked areas, the model is much better at predicting the correct class.

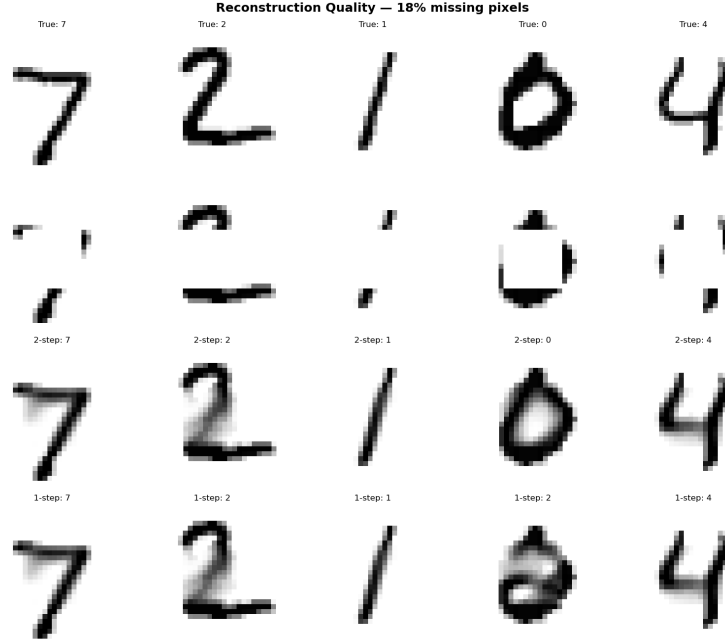


Figure 4: **Reconstruction at 18% censoring.** The structural integrity of the images is largely preserved. The images become easily recognizable, and the gap in accuracy between different reconstruction methods begins to narrow significantly.

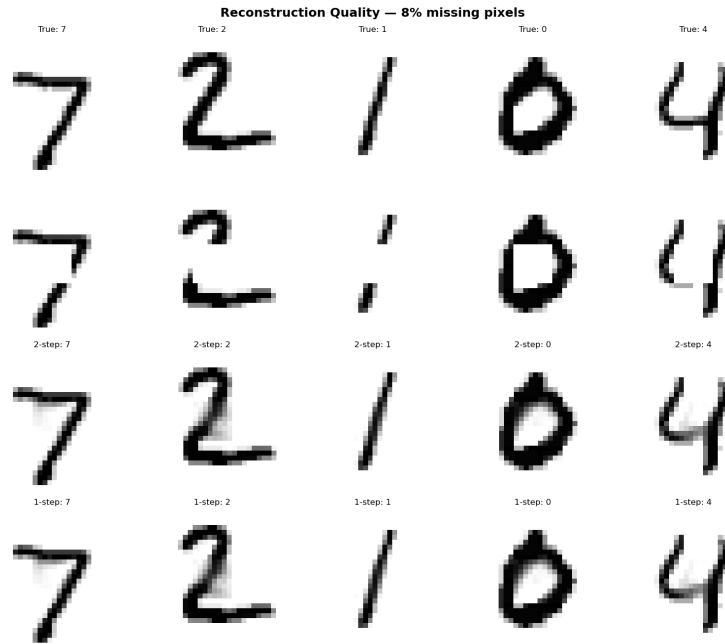


Figure 5: **Reconstruction at 8% censoring.** Reconstructions are clean and visually sharp. The missing regions are small enough that the surrounding visible pixels act as a tight guide for accurate filling.

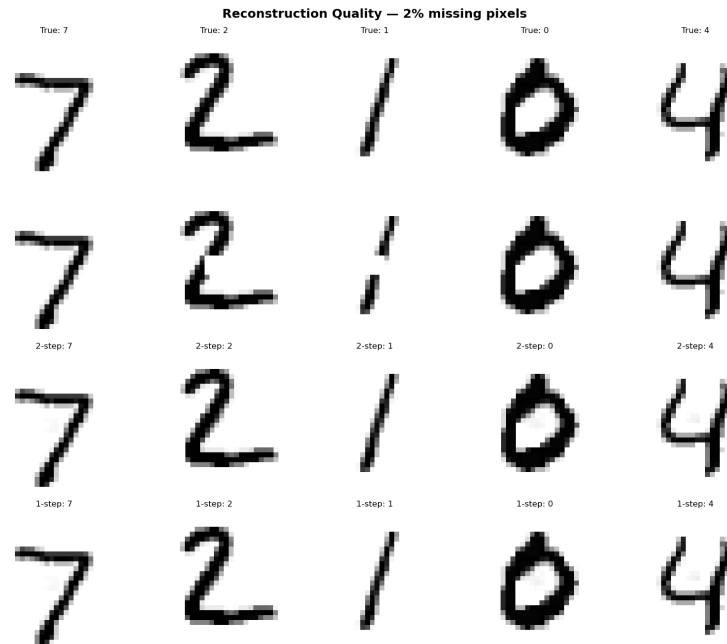


Figure 6: **Reconstruction at 2% censoring.** Reconstructed images are virtually indistinguishable from the uncensored originals, showing near pixel-perfect accuracy.

References

- [1] C. M. Bishop. Pattern Recognition and Machine Learning. Springer, 2006.
- [2] P. Kar. CS771 Lecture 15: Generative Models. IIT Kanpur, 2025–26.